

FLOWRA Production Deployment Guide

**Complete Step-by-Step: Digital Ocean + MongoDB + Domain +
SSL + Monitoring**

TABLE OF CONTENTS

1. Phase 1 — Buy Domain Name
 2. Phase 2 — Create Digital Ocean Account & Server
 3. Phase 3 — Point Domain to Server (DNS Setup)
 4. Phase 4 — First Login to Your Server (SSH)
 5. Phase 5 — Install All Software on Server
 6. Phase 6 — Secure MongoDB Database
 7. Phase 7 — Upload FLOWRA Code to Server
 8. Phase 8 — Configure Backend (Python/FastAPI)
 9. Phase 9 — Configure Frontend (React)
 10. Phase 10 — Configure Nginx (Web Server)
 11. Phase 11 — Enable HTTPS with SSL Certificate
 12. Phase 12 — Start the Application with PM2
 13. Phase 13 — Verify Everything Works
 14. Phase 14 — Automated Daily Database Backups
 15. Phase 15 — Server Monitoring & Alerts
 16. Phase 16 — Update Desktop Agent for Production
 17. Phase 17 — Ongoing Maintenance Commands
 18. Appendix A — Troubleshooting Common Issues
 19. Appendix B — Monthly Cost Summary
 20. Appendix C — Security Checklist
-

PHASE 1 — BUY DOMAIN NAME

Step 1.1: Choose a Domain Registrar

Go to any of these websites: - GoDaddy: <https://www.godaddy.com> (most popular in India) - Namecheap: <https://www.namecheap.com> (cheapest prices) - Google Domains: <https://domains.google> (simple interface) - BigRock: <https://www.bigrock.in> (Indian registrar)

Step 1.2: Search and Purchase

- Search for your domain name (e.g., "flowra.in")
- .in domains cost approximately Rs.800-1500 per year
- .com domains cost approximately Rs.900-1200 per year
- Complete the purchase with your payment details
- IMPORTANT: Keep your registrar login credentials safe

Step 1.3: Verify Domain Purchase

- You will receive a confirmation email
 - Log into your registrar dashboard
 - You should see your domain listed under "My Domains"
 - DO NOT change any DNS settings yet — we will do this in Phase 3
-

PHASE 2 — CREATE DIGITAL OCEAN ACCOUNT & SERVER

Step 2.1: Create Digital Ocean Account

- Go to <https://www.digitalocean.com>
- Click "Sign Up"
- Sign up with your email address
- Add a credit card or PayPal for billing
- NEW ACCOUNTS GET \$200 FREE CREDITS FOR 60 DAYS

Step 2.2: Create an SSH Key (on your local computer)

SSH keys let you securely connect to your server without typing passwords.

On Windows:

1. Open PowerShell (search "PowerShell" in Start menu)
2. Type this command and press Enter: `ssh-keygen -t ed25519 -C "your@email.com"`
3. Press Enter 3 times (accept default location, no passphrase)
4. Your key is saved. Now display it: `cat %env:USERPROFILE%.ssh\id_ed25519.pub`
5. COPY the entire output (starts with "ssh-ed25519...")

On Mac/Linux:

1. Open Terminal
2. Type: `ssh-keygen -t ed25519 -C "your@email.com"`
3. Press Enter 3 times
4. Display the key: `cat ~/.ssh/id_ed25519.pub`
5. COPY the entire output

Step 2.3: Create the Droplet (Server)

1. In Digital Ocean dashboard, click the green "Create" button (top right)

2. Select "Droplets"

Choose Region:

- Select "Bangalore" (BLR1) — this is closest to Indian users
- If Bangalore is not available, choose "Singapore" (SGP1)

Choose Image:

- Select "Ubuntu"
- Version: "24.04 (LTS)" — LTS means Long Term Support (most stable)

Choose Size:

- Click "Regular" (SSD)
- RECOMMENDED FOR LAUNCH: Select "\$24/mo"
- 2 vCPUs (processing cores)
- 4 GB RAM (memory)
- 80 GB SSD (storage)
- 4 TB transfer (bandwidth)
- This handles 50+ simultaneous users easily
- You can upgrade anytime with 1 click, zero downtime

Choose Authentication:

- Select "SSH Key"
- Click "New SSH Key"
- PASTE the SSH key you copied in Step 2.2
- Give it a name like "My Laptop"
- Click "Add SSH Key"

Choose Hostname:

- Type: flowra-production

Additional Options (Optional but Recommended):

- CHECK "Monitoring" (free — adds CPU/memory graphs)
- CHECK "IPv6" (free — future-proofing)

Create:

- Click "Create Droplet"
 - Wait 30-60 seconds for it to spin up
 - WRITE DOWN THE IP ADDRESS shown (e.g., 164.92.178.45) This is your server's public address.
-

PHASE 3 — POINT DOMAIN TO SERVER

(DNS SETUP)

Step 3.1: Go to Your Domain Registrar

- Log into GoDaddy/Namecheap/wherever you bought the domain
- Find "DNS Management" or "DNS Settings" for your domain

Step 3.2: Add DNS Records

Delete any existing A records, then add these:

Record 1 — Main Domain:

- Type: A
- Name: @ (this means flowra.in itself)
- Value: YOUR_DROPLET_IP (e.g., 164.92.178.45)
- TTL: 600 (or "10 minutes")

Record 2 — WWW Subdomain:

- Type: A
- Name: www
- Value: YOUR_DROPLET_IP (same IP)
- TTL: 600

Record 3 — API Subdomain (optional, for future use):

- Type: A
- Name: api
- Value: YOUR_DROPLET_IP (same IP)
- TTL: 600

Step 3.3: Verify DNS Propagation

- Wait 5-30 minutes

- Go to <https://dnschecker.org>
 - Type your domain (flowra.in)
 - Select record type "A"
 - Click "Search"
 - You should see your Droplet IP appearing across the world
 - If not all green yet, wait a few more minutes — it can take up to 1 hour
-

PHASE 4 — FIRST LOGIN TO YOUR SERVER (SSH)

Step 4.1: Connect via SSH

From Windows (PowerShell):

```
ssh root@164.92.178.45
```

(Replace with your actual Droplet IP)

From Mac/Linux (Terminal):

```
ssh root@164.92.178.45
```

Step 4.2: First Time Connection

- You will see a message: "The authenticity of host... Are you sure you want to continue?"
- Type: yes
- Press Enter
- You are now logged into your server!
- You should see a prompt like: root@flowra-production:~#

Step 4.3: Update the Server (ALWAYS do this first)

```
apt update && apt upgrade -y
```

- This updates all system packages to latest security patches
 - If asked "Which services should be restarted?" — press Enter (accept defaults)
 - If asked about a config file — press Enter (keep existing)
 - This may take 2-5 minutes
-

PHASE 5 — INSTALL ALL SOFTWARE ON SERVER

Run each section below one at a time. Wait for each to complete before moving to the next.

Step 5.1: Install Node.js 20 (for React frontend)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -  
apt install -y nodejs
```

Verify:

```
node --version
```

Should show: v20.x.x

Step 5.2: Install Yarn (frontend package manager)

```
npm install -g yarn
```

Verify:

```
yarn --version
```

Should show: 1.22.x

Step 5.3: Install Python 3.11+ (for FastAPI backend)

```
apt install -y python3 python3-pip python3-venv python3-dev build-essential
```

Verify:

```
python3 --version
```

Should show: Python 3.11.x or 3.12.x

Step 5.4: Install MongoDB 7 (database)

```
# Import MongoDB GPG key
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | gpg --dearmor -o /usr/share/keyrings/mongodb-server-7.0.gpg

# Add MongoDB repository
echo "deb [ signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0" | tee /etc/apt/sources.list.d/mongodb-org-7.0.list

# Install
apt update
apt install -y mongodb-org

# Start and enable (auto-start on boot)
systemctl enable mongod
systemctl start mongod
```

Verify:

```
systemctl status mongod
```

Should show: active (running)

Also test:

```
mongosh --eval "db.runCommand({ping:1})"
```

Should show: { ok: 1 }

Step 5.5: Install Nginx (web server / reverse proxy)

```
apt install -y nginx
systemctl enable nginx
systemctl start nginx
```

Verify:

```
systemctl status nginx
```

Should show: active (running)

Quick test — open your browser and go to: http://YOUR_DROPLET_IP (e.g., <http://164.92.178.45>) You should see the "Welcome to nginx!" page.

Step 5.6: Install Certbot (for free SSL certificates)

```
apt install -y certbot python3-certbot-nginx
```

Step 5.7: Install PM2 (process manager — keeps backend running 24/7)

```
npm install -g pm2
```

Verify:

```
pm2 --version
```

Step 5.8: Install Git

```
apt install -y git
```

Step 5.9: Configure Firewall

```
# Allow SSH (so you don't lock yourself out!)
ufw allow OpenSSH

# Allow web traffic
ufw allow 'Nginx Full'

# Enable firewall
ufw --force enable

# Verify
ufw status
```

Should show: SSH, Nginx Full — ALLOW

PHASE 6 — SECURE MONGODB DATABASE

Step 6.1: Create Database Admin User

```
mongosh
```

In the MongoDB shell, type:

```
use admin
db.createUser({
  user: "flowra_admin",
  pwd: "CHOOSE_A_STRONG_PASSWORD_HERE",
  roles: [
    { role: "userAdminAnyDatabase", db: "admin" },
    { role: "readWriteAnyDatabase", db: "admin" }
  ]
})
exit
```

WRITE DOWN this password. You will need it in Step 8.2.

Step 6.2: Enable MongoDB Authentication

```
nano /etc/mongod.conf
```

Find the section that says:

```
#security:
```

Change it to:

```
security:
  authorization: enabled
```

Also find the "bindIp" line and make sure it says:

```
bindIp: 127.0.0.1
```

(This ensures MongoDB ONLY accepts local connections — not from the internet)

Save the file: Press Ctrl+X, then Y, then Enter

Restart MongoDB:

```
systemctl restart mongod
```

Verify authentication works:

```
mongosh -u flowra_admin -p "YOUR_PASSWORD" --authenticationDatabase admin
```

You should get the MongoDB prompt. Type "exit" to leave.

PHASE 7 — UPLOAD FLOWRA CODE TO SERVER

Step 7.1: Create Application User (security best practice)

```
adduser flowra --disabled-password --gecos ""  
mkdir -p /home/flowra/app
```

Step 7.2: Upload Code

OPTION A — From GitHub (Recommended):

First, save your code to GitHub using the "Save to GitHub" button in Emergent. Then on the server:

```
cd /home/flowra/app  
git clone https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git .
```

OPTION B — Direct Upload from Your Computer:

On your LOCAL computer (not the server), open a new terminal:

```
scp -r /path/to/your/app/* root@YOUR_DROPLET_IP:/home/flowra/app/
```

Step 7.3: Set Correct Permissions

```
chown -R flowra:flowra /home/flowra/app
```

Step 7.4: Verify Files

```
ls -la /home/flowra/app/
```

You should see: backend/, frontend/, desktop-agent/, and other files.

PHASE 8 — CONFIGURE BACKEND

(Python/FastAPI)

Step 8.1: Create Python Virtual Environment

```
su - flowra
cd /home/flowra/app/backend

python3 -m venv venv
source venv/bin/activate
```

Your prompt should now show (venv) at the beginning.

Step 8.2: Install Python Dependencies

```
pip install --upgrade pip
pip install -r requirements.txt
pip install emergentintegrations --extra-index-url https://d33sy5i8bnduwe.cloudfront.net
```

This may take 3-5 minutes.

Step 8.3: Generate Security Keys

Run these two commands and SAVE the outputs:

```
# Generate JWT Secret (for login tokens)
python3 -c "import secrets; print('JWT_SECRET:', secrets.token_hex(32))"

# Generate Encryption Key (for database encryption)
python3 -c "from cryptography.fernet import Fernet; print('ENCRYPTION_KEY:', Fernet.generate_key().hex())"
```

COPY both values. You need them in the next step.

Step 8.4: Create Production .env File

```
nano /home/flowra/app/backend/.env
```

Paste this content (replace the placeholder values with YOUR actual values):

```
MONGO_URL=mongodb://flowra_admin:YOUR_MONGODB_PASSWORD@localhost:27017/flowra_production
DB_NAME=flowra_production
JWT_SECRET=PASTE_YOUR_JWT_SECRET_HERE
ENCRYPTION_KEY=PASTE_YOUR_ENCRYPTION_KEY_HERE
OPENAI_API_KEY=PASTE_YOUR_EMERGENT_LLM_KEY_HERE
```

IMPORTANT NOTES: - Replace YOUR_MONGODB_PASSWORD with the password from Step 6.1 - Replace JWT_SECRET with the value from Step 8.3 - Replace ENCRYPTION_KEY with the value from Step 8.3 - Replace OPENAI_API_KEY with your Emergent Universal Key - DO NOT add any spaces around the = signs - DO NOT add any comments in this file

Save: Ctrl+X, then Y, then Enter

Step 8.5: Test Backend Locally

```
cd /home/flowra/app/backend
source venv/bin/activate
python3 -c "from server import app; print('Backend imports OK')"
```

If you see "Backend imports OK" — the backend is configured correctly.

Quick run test:

```
uvicorn server:app --host 127.0.0.1 --port 8001 &
sleep 3
curl -s http://localhost:8001/api/auth/me | python3 -c "import sys,json; print(json.loads(sys.stdin.read()))"
kill %1
```

Then exit back to root:

```
exit
```

PHASE 9 — CONFIGURE FRONTEND (React)

Step 9.1: Create Production .env

```
su - flowra  
cd /home/flowra/app/frontend  
  
nano .env
```

Content (use YOUR domain):

```
REACT_APP_BACKEND_URL=https://flowra.in
```

Save: Ctrl+X, then Y, then Enter

Step 9.2: Install Dependencies and Build

```
cd /home/flowra/app/frontend  
yarn install
```

This may take 3-5 minutes.

```
yarn build
```

This may take 2-3 minutes. You should see "Compiled successfully" at the end.

Step 9.3: Verify Build

```
ls -la /home/flowra/app/frontend/build/
```

You should see: index.html, static/, and other files.

Exit back to root:

```
exit
```

PHASE 10 — CONFIGURE NGINX (Web Server)

Nginx serves your frontend and proxies API requests to the backend.

Step 10.1: Create Nginx Configuration

```
nano /etc/nginx/sites-available/flowra
```

Paste this ENTIRE content (replace flowra.in with YOUR domain):

```

# Redirect HTTP to HTTPS
server {
    listen 80;
    server_name flowra.in www.flowra.in;
    return 301 https://$server_name$request_uri;
}

# Main HTTPS server
server {
    listen 443 ssl http2;
    server_name flowra.in www.flowra.in;

    # SSL certificates (will be configured by Certbot in Phase 11)
    # ssl_certificate /etc/letsencrypt/live/flowra.in/fullchain.pem;
    # ssl_certificate_key /etc/letsencrypt/live/flowra.in/privkey.pem;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Max upload size (for future file uploads)
    client_max_body_size 50M;

    # Gzip compression (faster page loads)
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml text/javascript;
    gzip_min_length 1000;

    # Frontend – serve React build files
    location / {
        root /home/flowra/app/frontend/build;
        try_files $uri $uri/ /index.html;

        # Cache static assets (CSS, JS, images) for 1 year
        location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
            expires 1y;
            add_header Cache-Control "public, immutable";
        }
    }

    # Backend API – proxy to FastAPI
    location /api/ {
        proxy_pass http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
    }
}

```

```

        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 300s;
        proxy_connect_timeout 60s;
        proxy_send_timeout 300s;
    }

    # WebSocket support (for real-time sync updates)
    location /ws {
        proxy_pass http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 86400;
    }

    # Desktop agent download
    location /flowra-desktop-agent.py {
        root /home/flowra/app/frontend/build;
        add_header Content-Disposition "attachment; filename=flowra-desktop-agent.py";
    }
}

```

Save: Ctrl+X, then Y, then Enter

Step 10.2: Enable the Site

```

# Create symbolic link to enable
ln -sf /etc/nginx/sites-available/flowra /etc/nginx/sites-enabled/

# Remove default site
rm -f /etc/nginx/sites-enabled/default

```

Step 10.3: Test Nginx Configuration (IMPORTANT — always do this before restart)

```
nginx -t
```

You should see:


```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

If you see ANY errors, check your configuration file for typos.

Step 10.4: Temporarily Allow HTTP for SSL Setup

Before getting SSL, we need Nginx to listen on port 80 first. Edit the config temporarily:

```
nano /etc/nginx/sites-available/flowra
```

Replace the ENTIRE content with this TEMPORARY config:

```
server {
    listen 80;
    server_name flowra.in www.flowra.in;

    location / {
        root /home/flowra/app/frontend/build;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Save and restart:

```
nginx -t && systemctl restart nginx
```

PHASE 11 — ENABLE HTTPS WITH SSL CERTIFICATE

Step 11.1: Get SSL Certificate from Let's Encrypt

```
certbot --nginx -d flowra.in -d www.flowra.in
```

When prompted: - Enter your email address (for expiry notifications) - Type "Y" to agree to terms of service - Type "N" for sharing email with EFF (optional) - If asked about redirect, choose option 2 (redirect HTTP to HTTPS)

Certbot will: - Verify you own the domain - Generate SSL certificates - Automatically update your Nginx config - Set up auto-renewal (certificates renew every 90 days automatically)

Step 11.2: Now Restore the Full Nginx Config

```
nano /etc/nginx/sites-available/flowra
```

Now paste back the FULL configuration from Step 10.1 (the one with all the security headers, gzip, caching, etc.)

BUT this time, UNCOMMENT the SSL certificate lines (remove the # symbols):

```
ssl_certificate /etc/letsencrypt/live/flowra.in/fullchain.pem;  
ssl_certificate_key /etc/letsencrypt/live/flowra.in/privkey.pem;
```

Save and restart:

```
nginx -t && systemctl restart nginx
```

Step 11.3: Verify SSL

- Open browser: <https://flowra.in>

- You should see the padlock icon in the address bar
- The page should show your FLOWRA landing page

Step 11.4: Test Auto-Renewal

```
certbot renew --dry-run
```

Should show: "Congratulations, all renewals succeeded."

PHASE 12 — START THE APPLICATION WITH PM2

Step 12.1: Create PM2 Ecosystem File

```
su - flowra
cd /home/flowra/app/backend

cat > ecosystem.config.js << 'PMEOF'
module.exports = {
  apps: [{
    name: 'flowra-backend',
    script: '/home/flowra/app/backend/venv/bin/uvicorn',
    args: 'server:app --host 0.0.0.0 --port 8001 --workers 2',
    cwd: '/home/flowra/app/backend',
    env: {
      PATH: '/home/flowra/app/backend/venv/bin:/usr/local/bin:/usr/bin:/bin'
    },
    max_memory_restart: '500M',
    autorestart: true,
    watch: false,
    max_restarts: 10,
    restart_delay: 5000,
    log_date_format: 'YYYY-MM-DD HH:mm:ss Z',
    error_file: '/home/flowra/logs/backend-error.log',
    out_file: '/home/flowra/logs/backend-out.log',
    merge_logs: true
  }]
};
PMEOF
```

Step 12.2: Create Log Directory

```
mkdir -p /home/flowra/logs
```

Step 12.3: Start the Backend

```
cd /home/flowra/app/backend  
pm2 start ecosystem.config.js
```

You should see a table showing "flowra-backend" with status "online".

Step 12.4: Save PM2 Process List (survives reboot)

```
pm2 save
```

Step 12.5: Setup PM2 Auto-Start on Boot

```
pm2 startup
```

PM2 will print a command that starts with "sudo env PATH=..." COPY that exact command and run it (exit to root first if needed):

```
exit # back to root  
# PASTE AND RUN the command PM2 printed
```

Step 12.6: Verify Backend is Running

```
pm2 status
```

Should show: flowra-backend | online

```
pm2 logs flowra-backend --lines 10
```

Should show: "Application startup complete"

PHASE 13 — VERIFY EVERYTHING WORKS

Step 13.1: Test Backend API

```
curl -s https://flowra.in/api/auth/login \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"username":"superadmin","password":"superadmin123"}' | python3 -m json.tool
```

You should see: "success": true with login data.

Step 13.2: Test Frontend

- Open your browser
- Go to: <https://flowra.in>
- You should see the FLOWRA landing page
- Click "Login"
- Login with superadmin / superadmin123
- Verify the SuperAdmin dashboard loads

Step 13.3: Test HTTPS Redirect

- Go to: <http://flowra.in> (without https)
- It should automatically redirect to <https://flowra.in>

Step 13.4: Test from Mobile

- Open <https://flowra.in> on your phone
 - Verify the page loads correctly
 - Padlock icon should be visible
-

PHASE 14 — AUTOMATED DAILY DATABASE BACKUPS

Step 14.1: Create Backup Script

```
mkdir -p /home/flowra/backups

cat > /home/flowra/backup.sh << 'BACKUPEOF'
#!/bin/bash
# FLOWRA Daily Backup Script
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/home/flowra/backups"
LOG_FILE="$BACKUP_DIR/backup.log"

echo "[$TIMESTAMP] Starting backup..." >> $LOG_FILE

# Dump database
mongodump \
  --uri="mongodb://flowra_admin:YOUR_MONGODB_PASSWORD@localhost:27017/flowra_product" \
  --out="$BACKUP_DIR/backup_$TIMESTAMP" \
  2>> $LOG_FILE

if [ $? -eq 0 ]; then
  # Compress backup
  cd $BACKUP_DIR
  tar -czf "backup_$TIMESTAMP.tar.gz" "backup_$TIMESTAMP"
  rm -rf "backup_$TIMESTAMP"

  # Delete backups older than 30 days
  find $BACKUP_DIR -name "backup_*.tar.gz" -mtime +30 -delete

  SIZE=$(du -sh "backup_$TIMESTAMP.tar.gz" | cut -f1)
  echo "[$TIMESTAMP] Backup successful: backup_$TIMESTAMP.tar.gz ($SIZE)" >> $LOG_FILE
else
  echo "[$TIMESTAMP] BACKUP FAILED!" >> $LOG_FILE
fi
BACKUPEOF
```

IMPORTANT: Replace YOUR_MONGODB_PASSWORD with your actual MongoDB password.

Step 14.2: Make Executable and Set Permissions

```
chmod +x /home/flowra/backup.sh
chown flowra:flowra /home/flowra/backup.sh
chown -R flowra:flowra /home/flowra/backups
```

Step 14.3: Test Backup Manually

```
/home/flowra/backup.sh
ls -la /home/flowra/backups/
```

You should see a .tar.gz file.

Step 14.4: Schedule Daily Backup (2:00 AM IST = 8:30 PM UTC)

```
crontab -e
```

If asked which editor, choose "1" (nano).

Add this line at the bottom:

```
30 20 * * * /home/flowra/backup.sh
```

Save: Ctrl+X, then Y, then Enter

Step 14.5: How to Restore from Backup (in case of emergency)

```
# List available backups
ls -la /home/flowra/backups/

# Extract a backup
cd /home/flowra/backups
tar -xzf backup_20260411_203000.tar.gz

# Restore
mongorestore \
  --uri="mongodb://flowra_admin:YOUR_PASSWORD@localhost:27017/?authSource=admin" \
  --db flowra_production \
  --drop \
  backup_20260411_203000/flowra_production/
```

PHASE 15 — SERVER MONITORING & ALERTS

Step 15.1: Enable Digital Ocean Monitoring (Free)

- Go to Digital Ocean dashboard
- Click on your Droplet (flowra-production)
- Click the "Monitoring" tab on the left
- Click "Install Monitoring Agent" if not already installed:

```
bash curl -sSL https://repos.insights.digitalocean.com/install.sh | bash
```

Step 15.2: Set Up Alert Policies

In Digital Ocean dashboard: - Go to "Monitoring" in the left sidebar - Click "Create Alert Policy" - Create these alerts:

Alert 1 — High CPU:

- Resource: Droplet (flowra-production)
- Metric: CPU utilization
- Threshold: Above 80% for 5 minutes
- Notification: Your email

Alert 2 — High Memory:

- Metric: Memory utilization
- Threshold: Above 90% for 5 minutes

Alert 3 — High Disk:

- Metric: Disk utilization
- Threshold: Above 85% for 5 minutes

Step 15.3: Setup UptimeRobot (Free Website Monitoring)

- Go to <https://uptimerobot.com>

- Create a free account
- Click "Add New Monitor"

Monitor 1 — Website Up:

- Monitor Type: HTTPS
- Friendly Name: FLOWRA Website
- URL: <https://flowra.in>
- Monitoring Interval: 5 minutes

Monitor 2 — API Health:

- Monitor Type: HTTPS
- Friendly Name: FLOWRA API
- URL: <https://flowra.in/api/auth/me>
- Monitoring Interval: 5 minutes
- Add your email and/or phone for alerts
- UptimeRobot will email/SMS you within 5 minutes if the site goes down

Step 15.4: PM2 Log Rotation (prevent log files from filling disk)

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
pm2 set pm2-logrotate:compress true
```

PHASE 16 — UPDATE DESKTOP AGENT FOR PRODUCTION

Once you confirm <https://flowra.in> is live and working:

Step 16.1: Update Agent Default URL

In the desktop agent file, change the `BACKEND_URL` default:

```
BACKEND_URL = os.getenv('FLOWRA_BACKEND', 'https://flowra.in')
```

Step 16.2: Users Just Need to:

1. Download `flowra-desktop-agent.py` from the Setup page
2. Run: `python flowra-desktop-agent.py`
3. Enter their email and password
4. Agent auto-connects to <https://flowra.in> and starts syncing

(Tell me when you're ready and I will make this code change for you)

PHASE 17 — ONGOING MAINTENANCE

COMMANDS

Daily Checks (takes 30 seconds):

```
# Check everything is running
pm2 status
systemctl status mongod
systemctl status nginx

# Check disk usage
df -h /
```

View Logs:

```
# Backend logs (last 50 lines)
pm2 logs flowra-backend --lines 50

# Nginx access logs
tail -50 /var/log/nginx/access.log

# Nginx error logs
tail -50 /var/log/nginx/error.log

# MongoDB logs
tail -50 /var/log/mongodb/mongod.log
```

Restart Services:

```
# Restart backend
pm2 restart flowra-backend

# Restart Nginx
systemctl restart nginx

# Restart MongoDB
systemctl restart mongod
```

Deploy Code Updates:

```
su - flowra
cd /home/flowra/app

# Pull latest code
git pull origin main

# Update backend
cd backend
source venv/bin/activate
pip install -r requirements.txt
cd ..

# Rebuild frontend
cd frontend
yarn install
yarn build
cd ..

# Restart backend
pm2 restart flowra-backend

# Restart Nginx (to pick up new static files)
exit # back to root
systemctl restart nginx
```

Resize Server (if you need more power):

1. Go to Digital Ocean dashboard
2. Click on your Droplet

3. Click "Resize"
 4. Choose new size
 5. Click "Resize" — takes ~1 minute, zero data loss
-

APPENDIX A — TROUBLESHOOTING

Problem: "502 Bad Gateway" on the website

- Backend is not running
- Fix: pm2 restart flowra-backend
- Check: pm2 logs flowra-backend --lines 30

Problem: Cannot connect to MongoDB

- Check: systemctl status mongod
- Fix: systemctl restart mongod
- Verify: mongosh -u flowra_admin -p "PASSWORD" --authenticationDatabase admin

Problem: SSL certificate expired

- Fix: certbot renew
- Then: systemctl restart nginx

Problem: Server out of disk space

- Check: df -h /
- Clean old backups: find /home/flowra/backups -name "*.tar.gz" -mtime +7 -delete
- Clean old logs: pm2 flush

Problem: Cannot SSH into server

- Check if firewall is blocking: Go to Digital Ocean console (web-based terminal)
- Fix: ufw allow OpenSSH

Problem: Desktop agent cannot connect

- Verify backend is running: curl https://flowra.in/api/auth/me
- Check if HTTPS is working: curl -I https://flowra.in
- Check Nginx logs: tail -50 /var/log/nginx/error.log

APPENDIX B — MONTHLY COST SUMMARY

Item	Monthly Cost (INR)
Digital Ocean Droplet (4GB)	Rs.2,000
Domain (.in — yearly/12)	Rs.100
SSL Certificate (Let's Encrypt)	Free
MongoDB (self-hosted)	Free
UptimeRobot monitoring	Free
Digital Ocean monitoring	Free
PM2 (process manager)	Free
TOTAL	~Rs.2,100/month

APPENDIX C — SECURITY CHECKLIST

After deployment, verify all these are in place:

- ☐ SSH key authentication (no password login)
- ☐ Firewall enabled (UFW) — only ports 22, 80, 443 open
- ☐ MongoDB bound to 127.0.0.1 only (not accessible from internet)
- ☐ MongoDB authentication enabled
- ☐ HTTPS enforced (HTTP redirects to HTTPS)
- ☐ Security headers in Nginx (X-Frame-Options, etc.)
- ☐ .env file has restricted permissions (chmod 600)
- ☐ Application runs as non-root user (flowra)
- ☐ Daily automated backups configured
- ☐ Monitoring and alerts configured
- ☐ AES-256 encryption for PII data in database
- ☐ JWT tokens for API authentication

To restrict .env file permissions:

```
chmod 600 /home/flowra/app/backend/.env
chown flowra:flowra /home/flowra/app/backend/.env
```

DOCUMENT INFO

- Version: 1.0
 - Date: April 2026
 - Application: FLOWRA — Tally Prime Analytics Platform
 - Author: FLOWRA Development Team
 - For: Production Deployment on Digital Ocean
-

END OF GUIDE